

Pre-defense — speaking copy

Spoken lines only. Delivery notes, timings and anticipated questions are in transcript.pdf.

1. Title 0:19

Good morning. I'm Sakif Khan. This is my pre-defense on Agentic Graph Reasoning — knowledge graph navigation with verification before the answer is emitted. My supervisor is Dr. Sadia Sharmin.

2. The problem 0:32

Language models answer factual questions fluently — and just as fluently when they do not hold the fact. Fluency and factuality come out of the same mechanism, so nothing in the wording separates them. A system that is confidently wrong is worse than one that says it does not know.

3. What a hallucination is 0:38

First, the word. A hallucination is fluent, confident, and backed by no source the model can point to.

Two kinds, separated by what it takes to catch them. Factually wrong: you need the true answer already, which is what we are trying to produce. Unsupported: you need only what was retrieved, and we have that. I claim the second.

4. Where it comes from 0:31

Why does it happen? Not a bug that escaped testing — it comes out of how these models are built, so the response has to be architectural.

Three origins. The training data. The model itself, where knowledge sits spread across the weights with no index. And the prompt.

5. Which of the three can we act on? 0:32

Only the third; the other two need retraining. So this is a systems problem: control what goes in, and check what comes back out against it.

Multi-hop compounds it. A wrong first step is never revisited, so the chain proceeds from a false premise. That is where this thesis lives.

6. What a knowledge graph is

0:37

Two things about the graph, because they shape everything after.

Facts are stored as triples — head, relation, tail. Ada Lovelace, place of birth, London. A question needing two of those chained is multi-hop. The point of a graph is that every fact has an address, so a claim can be checked by looking for an edge.

7. The complication: mediator nodes

0:34

Freebase stores any fact with more than two participants as a node rather than an edge. “Rainn Wilson played Dwight in The Office” is one node joining three things. Nearly two thirds of this graph’s nodes are that kind, and they carry no name.

So nothing here is called “plays”. Hold onto that.

8. Where existing approaches stop

0:41

Four families, and each stops somewhere specific — the table walks left to right. The first three stop before reasoning begins, or at a radius fixed in advance. Agentic navigation does interleave retrieval and reasoning, which is the right move.

But all four share one gap: whatever the final generation call produces is what gets emitted. Nothing checks what the answer *asserts*.

9. What everyone else does

0:56

Five systems, and the column that matters is the last one.

The four prior systems differ in how they explore — a fixed radius, generated paths, beam search, adaptive planning. Some decompose the question, one backtracks. None checks what its own answer asserts against what it retrieved, before answering. That empty column is where this thesis sits.

One caution. Their published accuracies are higher than anything I am about to show, and not comparable: different backbones, different subsets, and full Freebase rather than the environment I built.

10. Research questions

0:53

Three questions.

RQ1: does agentic navigation actually improve multi-hop accuracy — and does the advantage grow with the number of hops?

RQ2: given a system that already navigates the graph, what does a claim-level check against the traversed triples contribute?

RQ3: which components earn their cost, in accuracy and in tokens?

One framing note. RQ2 asks what verification *contributes*, not whether it reduces hallucination. That’s deliberate. The answer has several parts, and a yes-or-no question would have hidden most of them.

11. AGR: an explicit state machine

1:08

AGR is an explicit state machine — not a prompt loop. Six nodes.

The planner decomposes the question into ordered sub-objectives. The explorer scores candidate edges and expands the frontier. The evaluator decides whether the sub-objective has been met. The backtracker undoes a bad expansion and bans the edge that caused it. The verifier is the contribution. The answerer emits only what survived.

Three cycles — the three arrows returning to the explorer — and all three are bounded by explicit budgets rather than by model behaviour. Every cycle passes through a router checking a monotone counter, so termination does not depend on the model cooperating.

12. Constrained tools

0:37

One agent, six nodes — and only three touch the graph. The planner links entities, the explorer asks for relations and neighbours, the verifier checks adjacency. Four operations, never Cypher. Every call is deterministic and logged with its arguments and result, so the traversal is a record — and that record is what the verification layer checks against.

13. The Structural Verification Layer

0:32

This is the contribution.

Before anything is emitted, the draft is split into atomic claims, each checked against the triples the agent actually traversed. What cannot be grounded is re-explored or dropped; what is grounded carries its triples back with the answer. So the system hedges rather than asserts.

14. What *structural* means — and what it does not

0:58

Now the bound.

Structural means the check is against the graph the agent walked, not the model’s opinion of its own output. It does not mean the check reads the relation: the first two routes test adjacency, either direction, so a mother claim survives on a child edge — the mediator problem from slide 7, as a limitation.

“Its evidence” is narrower too: only the walked-graph check attaches triples, and the log keeps a count. Slide 24 has both. And a claim can be true and still be the wrong answer.

15. One claim, three routes

1:00

This is the inside of the verifier — the node between the evaluator deciding to answer and the answer going out.

It drafts an answer and breaks it into atomic claims, one model call. Each claim then takes one of three routes to being called supported, ordered by cost: only the third spends a model call. Plus-evidence sits on the first route only.

The box at the bottom is how the node exits: no unsupported claims means grounded and the answer goes out; otherwise retry if the budget allows, or give up.

16. One question, end to end

1:08

One real question, all the way through — a run out of the committed records.

“Who plays Dwight in The Office.” The planner splits it in two and resolves The Office to a node. The explorer scores the available relations and takes the regular-cast one; the evaluator resolves Dwight Schrute. Second hop: the character relation, and the evaluator resolves Rainn Wilson and answers. The verifier splits the draft into two claims, both supported.

Six model calls, depth two, eighteen supporting triples.

And look at that second relation: the path runs *through* a mediator, which is why it is named for the appearance rather than for “plays”.

17. The environment and the question sets

0:59

The environment is a Freebase-derived graph: 2.6 million entities, 8.3 million triples, seven thousand distinct relations, importing in 36 seconds.

Questions: 400 each from WebQSP and ComplexWebQuestions. The important column is reachability — for every question I verified the gold answer is actually reachable here. 97 percent on WebQSP, 99.2 on CWQ.

That is the *ceiling* on every accuracy figure I’m about to show. It is also the answer to the caution two slides back: an environment this reachable is not full Freebase, so these numbers are not comparable with published ones.

18. Making the comparison fair

0:32

Five systems, on the slide: a parametric control, three retrieval baselines, and AGR.

All five run on one frozen backbone, at temperature zero, under the same 25-call budget, on the same questions, against the same graph. That is what lets me attribute differences to architecture rather than to model capacity.

19. What is *not* held equal

0:43

One thing is not equal, and I would rather say it than have it found. Think-on-Graph prunes from a narrower candidate set than AGR. That cuts against it, not for it — a thinner set is cheaper — so it cannot explain the clipping, but it does make its score a lower bound.

And the parametric control is there because these benchmarks may be partly memorised.

20. Main results

1:10

Here is the comparison.

AGR reaches 0.755 Hits@1 and 0.642 F1 on WebQSP, and 0.522 and 0.469 on ComplexWebQuestions — ahead of every baseline on both.

Two things beyond the top line. First, vector RAG on ComplexWebQuestions: 0.203, *below* the no-retrieval control at 0.307. One verbalised triple cannot contain a chain, so single-shot retrieval is worse there than not retrieving at all. GraphRAG sits beside it at 0.205, but its one-hop radius confounds the paradigm, so the claim rests on vector RAG.

Second, the cost columns: 4,511 tokens and 6.2 calls against Think-on-Graph's 3,615 and 12.8. More tokens, half the calls — and calls are what the budget meters.

21. RQ1: accuracy against hop count

1:02

The direct answer to RQ1, and it is a shape rather than a difference of means.

ComplexWebQuestions, on the right. AGR goes 0.46, 0.55, 0.57 as questions get harder — one hop, two, three or more. It is the only system on that dataset that ends above where it started. Three of the other four decay monotonically; Think-on-Graph falls and partially recovers, still 0.08 below its own one-hop score.

One caution about the left panel: on WebQSP the three-or-more stratum is n equals 4. I draw no conclusion from four questions, and neither does the thesis.

22. The caveat I want to raise myself

1:05

I want to raise this before you do.

Split the questions by whether Think-on-Graph finished inside the shared 25-call cap. On the ones it *finishes*, it is ahead of AGR — 0.852 against 0.788 on WebQSP, 0.629 against 0.607 on CWQ.

The entire aggregate margin comes from the questions it cannot finish, where it drops to 0.197 and 0.188. It gets clipped on 29 percent of WebQSP and 44 percent of CWQ.

So the honest claim is not that AGR reasons better per step. It is that AGR completes its reasoning inside a fixed budget and Think-on-Graph frequently does not.

23. RQ2: does anything ungrounded get asserted?1:03

RQ2. Does anything ungrounded get asserted?

Pooling both datasets: AGR asserts 1,709 entities and zero are absent from the graph. The parametric control asserts 1,001 and 22.1 percent don't exist.

That looks like a headline result for the verification layer. It isn't, and this is the finding I think matters most: Think-on-Graph also reaches 0.0 percent, and it has no verification layer at all.

So zero ungrounded assertion is a property of *navigating a graph* — you can only name what you visited. It is not a property of my layer, and I don't claim it as one.

24. RQ2: what verification does *not* do0:53

The negative half first: removing the layer changes accuracy by an amount I cannot detect — p equals 1.0 on both datasets. And its evidence does not outlive the run: the logger keeps the count of supporting triples and drops the list.

This is not a retraction of slide 20. That was AGR against four baselines; this is AGR against itself with one part removed. The lead came from navigating the graph and finishing inside the budget — never from this layer.

25. RQ2: so what does it do?0:41

What it does do is withhold what it cannot ground. Removing the layer drops the CWQ hedge rate from 23.2 to 20.2 percent — direction only, six questions. A hedge is a calibrated non-assertion, not a refusal.

It attaches supporting triples, from one route of three, and pairs the answer with its evidence inside the run. So the case rests on auditability, not accuracy.

26. RQ3: what each component earns0:22

RQ3: which components earn their cost.

Four ablations, paired McNemar against the full system. Three of the four change nothing I can detect — backtracking, the verifier, and the learned half of the scorer.

27. RQ3: one effect, and its sign is backwards0:48

The one that does is the planner, and the sign is backwards. Removing it *improves* WebQSP by 0.083 F1 at p equals 0.006, and cuts tokens by 31 percent. On ComplexWebQuestions it trends the other way.

The explanation is hop count: WebQSP is mostly one-hop, and decomposing a one-hop question sends the frontier away from the answer. So decomposition should be gated on question structure — a conclusion the measurement forced, not one it confirmed.

28. Every failure, read and labelled

0:29

Two hundred and fifty-nine failures, read and labelled by hand: every AGR failure over the 800 test questions, less the adjudicated benchmark defects.

The largest category is relation selection — the agent taking the wrong edge, not inventing one. That shape is the finding.

29. The echo attractor

1:12

The one to name is sixth, with 13 cases: the **echo attractor** — a real, grounded entity one hop from the correct answer. Ask for a director and get the film; ask for a capital and get the country.

Why it matters, and it is the promise I made on slide 14: any grounding check *passes* it. It is real, it was traversed, the triple exists — true, and wrong.

Different systems fall into it together, so no evaluation treating them as independent can see it. Rescore whenever a majority agrees — a natural thing to want — and this becomes apparent correctness. That is the contribution: the mechanism, not the count.

30. Contributions, and what I would not claim

1:29

To summarise. Six contributions, the six the thesis claims in section 1.6. The one I'd underline is stratum-dependent decomposition: the literature treats it as straightforwardly beneficial, and it is not.

The sixth is worded *pre-registered* in the thesis; nothing was filed with a registry, so I say *pre-specified*.

The limitations, which I'd rather state than be asked. The first is the most serious: the verifier persists only what it *rejects*, so wrongful acceptance has no rate at all — and the same decision is why the output contract cannot be audited from the record.

Then: no detectable accuracy gain, and at n around 200 that is “not detected”, not “none”. Zero ungrounded assertion comes from navigation, not my layer. One environment, one backbone, one annotator. And Think-on-Graph leads where it finishes, from a narrower candidate set.

31. Thank you

0:05

Thank you. I'm happy to take questions.